

To get started with Entity Framework 6 Code First using ASP.NET MVC 5, you can follow a structured approach that includes setting up your project, creating your data model, and initializing your database. Below is a comprehensive guide based on the search results and general knowledge.

Step-by-Step Tutorial

1. Create an MVC Web Application

- Open **Visual Studio** and create a new project.
- Select **ASP.NET Web Application** and choose **MVC** as the project template.

2. Install Entity Framework 6

- Open the **Package Manager Console** from the **Tools** menu.
- Run the following command to install Entity Framework:

```
powershell
VerifyOpen In EditorRunCopy code
1Install-Package EntityFramework
```

3. Create the Data Model

- In the **Models** folder, create three classes: Student, Course, and Enrollment.

Student.cs

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Collections.Generic;
3
4namespace ContosoUniversity.Models
5{
6    public class Student
7    {
8        public int ID { get; set; }
9        public string LastName { get; set; }
10        public string FirstMidName { get; set; }
11        public DateTime EnrollmentDate { get; set; }
12
13        public virtual ICollection<Enrollment> Enrollments { get; set; }
14    }
15}
```

Course.cs

```
csharp
VerifyOpen In EditorRunCopy code
1using System.Collections.Generic;
2using System.ComponentModel.DataAnnotations.Schema;
3
4namespace ContosoUniversity.Models
5{
6    public class Course
7    {
8        [DatabaseGenerated(DatabaseGeneratedOption.None)]
9        public int CourseID { get; set; }
10        public string Title { get; set; }
11        public int Credits { get; set; }
12    }
13}
```

```

12
13     public virtual ICollection<Enrollment> Enrollments { get; set; }
14 }
15}

```

Enrollment.cs

```

csharp
VerifyOpen In EditorRunCopy code
1namespace ContosoUniversity.Models
2{
3    public enum Grade
4    {
5        A, B, C, D, F
6    }
7
8    public class Enrollment
9    {
10        public int EnrollmentID { get; set; }
11        public int CourseID { get; set; }
12        public int StudentID { get; set; }
13        public Grade? Grade { get; set; }
14
15        public virtual Course Course { get; set; }
16        public virtual Student Student { get; set; }
17    }
18}

```

4. Create the Database Context

- In the **DAL** (Data Access Layer) folder, create a class named `SchoolContext.cs`.

SchoolContext.cs

```

csharp
VerifyOpen In EditorRunCopy code
1using ContosoUniversity.Models;
2using System.Data.Entity;
3using System.Data.Entity.ModelConfiguration.Conventions;
4
5namespace ContosoUniversity.DAL
6{
7    public class SchoolContext : DbContext
8    {
9        public SchoolContext() : base("SchoolContext") { }
10
11        public DbSet<Student> Students { get; set; }
12        public DbSet<Enrollment> Enrollments { get; set; }
13        public DbSet<Course> Courses { get; set; }
14
15        protected override void OnModelCreating(DbModelBuilder modelBuilder)
16        {
17            modelBuilder.Conventions.Remove<PluralizingTableNameConvention>();
18        }
19    }
20}

```

5. Initialize the Database with Test Data

- Create a new class named `SchoolInitializer.cs` in the **DAL** folder.

SchoolInitializer.cs

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Collections.Generic;
3using System.Data.Entity;
4using ContosoUniversity.Models;
5
6namespace ContosoUniversity.DAL
7{
8    public class SchoolInitializer :
DropCreateDatabaseIfModelChanges<SchoolContext>
9    {
10        protected override void Seed(SchoolContext context)
11        {
12            var students = new List<Student>
13            {
14                new Student { FirstMidName = "Carson", LastName = "Alexander",
EnrollmentDate = DateTime.Parse("2005-09-01") },
15                new Student { FirstMidName = "Meredith", LastName = "Alonso",
EnrollmentDate = DateTime.Parse("2002-09-01") },
16                // Add more students as needed
17            };
18
19            students.ForEach(s => context.Students.Add(s));
20            context.SaveChanges();
21
22            var courses = new List<Course>
23            {
24                new Course { CourseID = 1050, Title = "Chemistry", Credits = 3
25                },
26                new Course { CourseID = 4022, Title = "Microeconomics",
Credits = 3 },
27                // Add more courses as needed
28            };
29            courses.ForEach(c => context.Courses.Add(c));
30            context.SaveChanges();
31
32            var enrollments = new List<Enrollment>
33            {
34                new Enrollment { StudentID = 1, CourseID = 1050, Grade =
Grade.A },
35                new Enrollment { StudentID = 1, CourseID = 4022, Grade =
Grade.C },
36                // Add more enrollments as needed
37            };
38
39            enrollments.ForEach(e => context.Enrollments.Add(e));
40            context.SaveChanges();
41        }
42    }
43}

```

6. Configure the Database Initializer

- In the `Global.asax.cs` file, set the database initializer in the `Application_Start` method:

```

csharp
VerifyOpen In EditorRunCopy code
1protected void Application_Start()
2{
3    Database.SetInitializer(new SchoolInitializer());

```

```
4 AreaRegistration.RegisterAllAreas();
5 RouteConfig.RegisterRoutes(RouteTable.Routes);
6}
```

7. Update the Web.config

- Ensure your connection string is set up in the Web.config file:

```
xml
VerifyOpen In EditorRunCopy code
1<connectionStrings>
2  <add name="SchoolContext" connectionString="Data Source=(localdb)\
MSSQLLocalDB;Initial Catalog=SchoolContext;Integrated Security=True"
providerName="System.Data.SqlClient" />
3</connectionStrings>
```

8. Run the Application

- Press **Ctrl + F5** to run the application. The database will be created, and the test data will be seeded.

Conclusion

This tutorial provides a foundational understanding of how to set up Entity Framework 6 with Code First in an ASP.NET MVC 5 application. You can expand upon this by creating controllers and views to manage your data effectively. For further learning, consider exploring CRUD operations and more advanced features of Entity Framework.

Introduction to Razor Pages in C# .NET

Razor Pages is a server-side web application framework that is part of ASP.NET Core. It is designed to make it easier to build dynamic, data-driven web applications with a clean separation of concerns. Here's a comprehensive overview to get you started with Razor Pages.

What is Razor Pages?

- **Page-Focused Framework:** Razor Pages is built around the concept of pages, making it intuitive for developers familiar with page-centric frameworks like PHP or ASP.NET Web Forms.
- **Separation of Concerns:** It encourages a clear separation between the UI and the business logic, similar to the MVC (Model-View-Controller) pattern.
- **Cross-Platform:** It supports development on Windows, macOS, and Linux.

Key Features

- **Simplified Routing:** Razor Pages uses a convention-based routing system that maps URLs to page files in the /Pages directory.
- **Built-in Model Binding:** Automatically binds form data to C# objects, reducing boilerplate code.
- **Validation:** Supports data validation using Data Annotations, ensuring that user input is validated both on the client and server sides.

- **Tag Helpers:** Provides a way to create HTML elements with server-side code, making it easier to work with forms and other UI elements.

Getting Started with Razor Pages

1. Create a New Razor Pages Project

You can create a new Razor Pages project using the .NET CLI or Visual Studio.

Using .NET CLI:

```
bash
VerifyOpen In EditorRunCopy code
1dotnet new webapp -o RazorPagesMovie
2cd RazorPagesMovie
3code -r .
```

Using Visual Studio:

- Open Visual Studio and select **Create a new project**.
- Choose **ASP.NET Core Web App (Razor Pages)**.
- Name your project (e.g., RazorPagesMovie) and click **Create**.

2. Project Structure

A typical Razor Pages project has the following structure:

```
VerifyOpen In EditorRunCopy code
1/Pages
2    /Customers
3        Create.cshtml
4        Edit.cshtml
5        Index.cshtml
6    _ViewImports.cshtml
7    _ViewStart.cshtml
8    Index.cshtml
9    Privacy.cshtml
```

- **.cshtml files:** Contain HTML markup and Razor syntax.
- **.cshtml.cs files:** Contain the C# code that handles page events.

3. Example of a Razor Page

Here's a simple example of a Razor Page that allows users to create a new customer.

Create.cshtml

```
razor
VerifyOpen In EditorRunCopy code
1@page
2@model RazorPagesContacts.Pages.Customers.CreateModel
3@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
4
5<h2>Create Customer</h2>
6
7<form method="post">
8    <div>
9        <label asp-for="Customer.Name"></label>
10       <input asp-for="Customer.Name" />
11       <span asp-validation-for="Customer.Name"></span>
```

```
12     </div>
13     <button type="submit">Create</button>
14</form>
```

Create.cshtml.cs

```
csharp
VerifyOpen In EditorRunCopy code
1using Microsoft.AspNetCore.Mvc;
2using Microsoft.AspNetCore.Mvc.RazorPages;
3using RazorPagesContacts.Data;
4using RazorPagesContacts.Models;
5using System.Threading.Tasks;
6
7namespace RazorPagesContacts.Pages.Customers
8{
9    public class CreateModel : PageModel
10    {
11        private readonly CustomerDbContext _context;
12
13        public CreateModel(CustomerDbContext context)
14        {
15            _context = context;
16        }
17
18        [BindProperty]
19        public Customer Customer { get; set; }
20
21        public IActionResult OnGet()
22        {
23            return Page();
24        }
25
26        public async Task<IActionResult> OnPostAsync()
27        {
28            if (!ModelState.IsValid)
29            {
30                return Page();
31            }
32
33            _context.Customers.Add(Customer);
34            await _context.SaveChangesAsync();
35
36            return RedirectToPage("./Index");
37        }
38    }
39}
```

Running the Application

- To run the application, use the command:

```
bash
VerifyOpen In EditorRunCopy code
1dotnet run
```

- Open your browser and navigate to <https://localhost:5001> to see your Razor Pages application in action.

Conclusion

Razor Pages is a powerful framework for building web applications in ASP.NET Core. It simplifies the development process by providing a page-centric approach, built-in model binding, and validation. As you continue to explore Razor Pages, you can leverage its features to create robust and maintainable web applications.

Additional Resources

- [Get started with Razor Pages](#)
- [ASP.NET Core documentation](#)

Feel free to ask if you have any specific questions or need further examples!

Final Answer

To develop an MVC application using the Entity Framework (EF) Code First approach and to understand Razor Pages in C# .NET, we can break down the concepts into detailed sections. Below is a comprehensive guide that covers both topics, including important points, examples, and explanations.

Developing MVC Application using EF Code First Approach

1. Overview of MVC and EF Code First

- **MVC (Model-View-Controller):** A design pattern that separates an application into three main components:
 - **Model:** Represents the data and business logic.
 - **View:** Represents the UI components.
 - **Controller:** Handles user input and interacts with the model to render the view.
- **Entity Framework (EF) Code First:** A method of using EF where you define your data model using C# classes, and EF generates the database schema based on these classes.

2. Step-by-Step Guide to Create an MVC Application with EF Code First

Step 1: Create an MVC Web Application

- Open **Visual Studio**.
- Select **File > New > Project**.
- Choose **ASP.NET Web Application** and select the **MVC** template.

Step 2: Install Entity Framework

- Open the **Package Manager Console**.
- Run the command:

```
powershell  
VerifyOpen In EditorRunCopy code  
1Install-Package EntityFramework
```

Step 3: Create the Data Model

- In the **Models** folder, create classes for your data entities. For example, Student, Course, and Enrollment.

Example Classes:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Student
2{
3    public int ID { get; set; }
4    public string LastName { get; set; }
5    public string FirstMidName { get; set; }
6    public DateTime EnrollmentDate { get; set; }
7    public virtual ICollection<Enrollment> Enrollments { get; set; }
8}
```

Step 4: Create the Database Context

- Create a class that inherits from DbContext to manage entity objects during runtime.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public class SchoolContext : DbContext
2{
3    public DbSet<Student> Students { get; set; }
4    public DbSet<Course> Courses { get; set; }
5    public DbSet<Enrollment> Enrollments { get; set; }
6}
```

Step 5: Initialize the Database with Test Data

- Use a database initializer to seed the database with initial data.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1public class SchoolInitializer :
DropCreateDatabaseIfModelChanges<SchoolContext>
2{
3    protected override void Seed(SchoolContext context)
4    {
5        // Seed data
6    }
7}
```

Step 6: Configure the Database Initializer

- In Global.asax.cs, set the database initializer:

```
csharp
VerifyOpen In EditorRunCopy code
1protected void Application_Start()
2{
3    Database.SetInitializer(new SchoolInitializer());
4}
```

Step 7: Update the Web.config

- Ensure your connection string is correctly set in `Web.config`:

```
xml
VerifyOpen In EditorRunCopy code
1<connectionStrings>
2    <add name="SchoolContext" connectionString="Data Source=(localdb)\
MSSQLLocalDB;Initial Catalog=SchoolContext;Integrated Security=True"
providerName="System.Data.SqlClient" />
3</connectionStrings>
```

Step 8: Run the Application

- Press **Ctrl + F5** to run the application. The database will be created, and the test data will be seeded.

3. Important Points to Remember

- **Code First Migrations:** Use migrations to update the database schema as your model changes.
 - **Data Annotations:** Use attributes like `[Required]`, `[StringLength]` for validation.
 - **Lazy Loading:** EF supports lazy loading, which loads related data on demand.
-

Introduction to Razor Pages in C# .NET

1. Overview of Razor Pages

- **Razor Pages:** A page-focused framework in ASP.NET Core that simplifies the development of web applications.
- **Separation of Concerns:** Encourages a clear separation between UI and business logic.

2. Key Features of Razor Pages

- **Simplified Routing:** Convention-based routing maps URLs to page files.
- **Built-in Model Binding:** Automatically binds form data to C# objects.
- **Validation:** Supports data validation using Data Annotations.
- **Tag Helpers:** Simplifies the creation of HTML elements with server-side code.

3. Getting Started with Razor Pages

Step 1: Create a New Razor Pages Project

- Use the .NET CLI or Visual Studio to create a new project.

Using .NET CLI:

```
bash
VerifyOpen In EditorRunCopy code
1dotnet new webapp -o RazorPagesMovie
2cd RazorPagesMovie
```

Using Visual Studio:

- Select **ASP.NET Core Web App (Razor Pages)**.

Step 2: Project Structure

- The project typically contains a `/Pages` directory with `.cshtml` files for UI and `.cshtml.cs` files for C# code.

Step 3: Example of a Razor Page

Create.cshtml

```
razor
VerifyOpen In EditorRunCopy code
1@page
2@model RazorPagesContacts.Pages.Customers.CreateModel
3
4<h2>Create Customer</h2>
5<form method="post">
6    <input asp-for="Customer.Name" />
7    <button type="submit">Create</button>
8</form>
```

Create.cshtml.cs

```
csharp
VerifyOpen In EditorRunCopy code
1public class CreateModel : PageModel
2{
3    public async Task<IActionResult> OnPostAsync()
4    {
5        // Handle form submission
6    }
7}
```

4. Running the Application

- Use the command:

```
bash
VerifyOpen In EditorRunCopy code
1dotnet run
```

- Navigate to `https://localhost:5001` to see your Razor Pages application.

5. Conclusion

- Razor Pages provides a streamlined approach to building web applications, focusing on pages rather than controllers and views. It simplifies development with built-in features like model binding and validation.

6. Additional Resources

- Razor Pages Documentation
-

This comprehensive guide should help you understand both the MVC application development using EF Code First and the Razor Pages framework in C#. If you have any specific questions or need further clarification on any topic, feel free to ask!